

Algorithms and complexity

Chapter 2

1

What is an algorithm?

- An *algorithm* is a sequence of instructions that one must perform in order to solve a well-formulated *problem*.
- *Pseudocode*. Language computer scientists often use to describe algorithms.
- Pseudocode's elementary operations:

- **Assignment**

$b \leftarrow 2$

- **Arithmetic**

$2 + b$

- **Conditional**

```
if  $a < b$ 
  return  $b$ 
else
  return  $a$ 
```

- **for loops**

```
for  $i \leftarrow 1$  to  $n$ 
   $sum \leftarrow sum + i$ 
```

- **while loops**

```
while  $total < b$ 
   $total \leftarrow total + i$ 
```

- **Array access**

```
for  $i \leftarrow 3$  to  $n$ 
   $F_i \leftarrow F_{i-1} + F_{i-2}$ 
```

2

Pseudo code's elementary operations

Assignment

Format: $a \leftarrow b$

Effect: Sets the variable a to the value b .

Example: $b \leftarrow 2$

$a \leftarrow b$

Result: The value of a is 2

Arithmetic

Format: $a + b, a - b, a \cdot b, a/b, a^b$

Effect: Addition, subtraction, multiplication, division, and exponentiation of numbers.

Example: $\text{DIST}(x1, y1, x2, y2)$
1 $dx \leftarrow (x2 - x1)^2$
2 $dy \leftarrow (y2 - y1)^2$
3 **return** $\sqrt{dx + dy}$

Result: $\text{DIST}(x1, y1, x2, y2)$ computes the Euclidean distance between points with coordinates $(x1, y1)$ and $(x2, y2)$. $\text{DISTANCE}(0, 0, 3, 4)$ returns 5.

8/29/21

Chapter 2. Algorithms and Complexity

3

3

Pseudo code's elementary operations (cont'd)

Conditional

Format: **if** A is true
 B
 else
 C

Effect: If statement A is true, executes instructions B , otherwise executes instructions C . Sometimes we will omit "**else** C ," in which case this will either execute B or not, depending on whether A is true.

Example: $\text{MAX}(a, b)$
1 **if** $a < b$
2 **return** b
3 **else**
4 **return** a

Result: $\text{MAX}(a, b)$ computes the maximum of the numbers a and b . For example, $\text{MAX}(1, 99)$ returns 99.

8/29/21

Chapter 2. Algorithms and Complexity

4

4

Pseudo code's elementary operations (cont'd)

for loops

Format: **for** $i \leftarrow a$ **to** b
 B

Effect: Sets i to a and executes instructions **B**. Sets i to $a + 1$ and executes instructions **B** again. Repeats for $i = a + 2, a + 3, \dots, b - 1, b$.³

Example: SUMINTEGERS(n)
1 $sum \leftarrow 0$
2 **for** $i \leftarrow 1$ **to** n
3 $sum \leftarrow sum + i$
4 **return** sum

Result: SUMINTEGERS(n) computes the sum of integers from 1 to n . SUMINTEGERS(10) returns $1 + 2 + \dots + 10 = 55$.

Pseudo code's elementary operations (cont'd)

while loops

Format: **while** A is true
 B

Effect: Checks the condition A . If it is true, then executes instructions **B**. Checks A again; if it's true, it executes **B** again. Repeats until A is not true.

Example: ADDUNTIL(b)
1 $i \leftarrow 1$
2 $total \leftarrow i$
3 **while** $total \leq b$
4 $i \leftarrow i + 1$
5 $total \leftarrow total + i$
6 **return** i

Result: ADDUNTIL(b) computes the smallest integer i such that $1 + 2 + \dots + i$ is larger than b . For example, ADDUNTIL(25) returns 7, since $1 + 2 + \dots + 7 = 28$, which is larger than 25, but $1 + 2 + \dots + 6 = 21$, which is smaller than 25.

Pseudo code's elementary operations (cont'd)

Array access

Format: a_i

Effect: The i th number of array $a = (a_1, \dots, a_i, \dots, a_n)$. For example, if $F = (1, 1, 2, 3, 5, 8, 13)$, then $F_3 = 2$, and $F_4 = 3$.

Example: FIBONACCI(n)
1 $F_1 \leftarrow 1$
2 $F_2 \leftarrow 1$
3 **for** $i \leftarrow 3$ **to** n
4 $F_i \leftarrow F_{i-1} + F_{i-2}$
5 **return** F_n

Result: FIBONACCI(n) computes the n th Fibonacci number. FIBONACCI(8) returns 21.

8/29/21

Chapter 2. Algorithms and Complexity

7

7

Informal language

1 $\frac{1}{2}$ cups canned or cooked pumpkin
1 cup brown sugar, firmly packed
 $\frac{1}{2}$ teaspoon salt
2 teaspoons cinnamon
1 teaspoon ginger
2 tablespoons molasses
3 eggs, slightly beaten
12 ounce can of evaporated milk
1 unbaked pie crust

Combine pumpkin, sugar, salt, ginger, cinnamon, and molasses. Add eggs and milk and mix thoroughly. Pour into unbaked pie crust and bake in hot oven (425 degrees Fahrenheit) for 40 to 45 minutes, or until knife inserted comes out clean.

8/29/21

Chapter 2. Algorithms and Complexity

8

8

Pseudocode

```
MAKEPUMPKINPIE(pumpkin, sugar, salt, spices, eggs, milk, crust)
1  PREHEATOVEN(425)
2  filling ← MIXFILLING(pumpkin, sugar, salt, spices, eggs, milk)
3  pie ← ASSEMBLE(crust, filling)
4  while knife inserted does not come out clean
5    BAKE(pie)
6  output "Pumpkin pie is complete"
7  return pie

MIXFILLING(pumpkin, sugar, salt, spices, eggs, milk)
1  bowl ← Get a bowl from cupboard
2  PUT(pumpkin, bowl)
3  PUT(sugar, bowl)
4  PUT(salt, bowl)
5  PUT(spices, bowl)
6  STIR(bowl)
7  PUT(eggs, bowl)
8  PUT(milk, bowl)
9  STIR(bowl)
10 filling ← Contents of bowl
11 return filling
```

8/29/21

Chapter 2. Algorithms and Complexity

9

9

Biological versus computer algorithms

- **DNA replication** *Before a cell can divide, it must first make a complete copy of all its genetic material.*
 1. Pair of complementary strands of DNA
 2. DNA helicase binds to the DNA
 - Enzyme that uses the hydrolysis of atp to unwind the dna helix at the replication fork, to allow the resulting single strands to be copied.
 3. Helicase wrenches apart the two strands of DNA
 4. Primers, which are short single strands of RNA, latch on to specific positions in the newly open strands.

8/29/21

Chapter 2. Algorithms and Complexity

10

10

DNA replication (cont'd)

5. A DNA polymerase binds to each separated template strand of the DNA.
6. At each nucleotide, DNA polymerase matches the template strand's nucleotide with the complementary base.
7. DNA ligase repairs the gaps in the newly synthesized DNA's backbone.
8. The original strands separate so that two pairs of DNA strands are formed.

8/29/21

Chapter 2. Algorithms and Complexity

11

11

DNA replication (cont'd)



8/29/21

Chapter 2. Algorithms and Complexity

12

12

String duplication problem

String duplication problem:

Given a string of letters, return a copy.

Input: A string $\mathbf{s}=(s_1, s_2, \dots, s_n)$ of length n , as an array of characters.

Output: A string representing the copy of $\mathbf{s}$.

8/29/21

Chapter 2. Algorithms and Complexity

13

13

The StringCopy algorithm

StringCopy($\mathbf{s}, n$)

1. for $i \leftarrow 1$ to n
2. $t_i \leftarrow s_i$
3. return $\mathbf{t}$

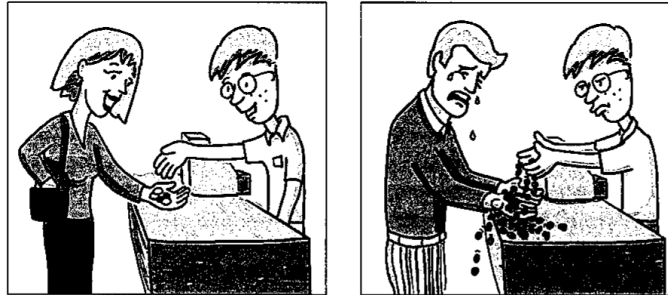
8/29/21

Chapter 2. Algorithms and Complexity

14

14

The Change Problem



8/29/21

Chapter 2. Algorithms and Complexity

15

15

The Change Problem

Change Problem:

Convert some amount of money M into given denominations, using the smallest possible number of coins.

Input: An amount of money M , and an array of d denominations $\mathbf{c} = (c_1, c_2, \dots, c_d)$, in decreasing order of value ($c_1 > c_2 > \dots > c_d$).

Output: A list of d integers $i_1, i_2, \dots, i_d$ such that $c_1 i_1 + c_2 i_2 + \dots + c_d i_d = M$, and $i_1 + i_2 + \dots + i_d$ is as small as possible.

8/29/21

Chapter 2. Algorithms and Complexity

16

16

United States Change Problem

United States Change Problem:

Convert some amount of money into the fewest number of coins.

Input: An amount of money, M , in cents.

Output: The smallest number of quarters q , dimes d , nickels n , and pennies p whose values add to M (i.e., $25q + 10d + 5n + p = M$ and $q + d + n + p$ is as small as possible).

US Change algorithm (1)

```
USCHANGE( $M$ )  
1  while  $M > 0$   
2     $c \leftarrow$  Largest coin that is smaller than (or equal to)  $M$   
3    Give coin with denomination  $c$  to customer  
4     $M \leftarrow M - c$ 
```

US Change algorithm (2)

```
USCHANGE( $M$ )
1  Give the integer part of  $M/25$  quarters to customer.
2  Let remainder be the remaining amount due the customer.
3  Give the integer part of remainder/10 dimes to customer.
4  Let remainder be the remaining amount due the customer.
5  Give the integer part of remainder/5 nickels to customer.
6  Let remainder be the remaining amount due the customer.
7  Give remainder pennies to customer.
```

Correct versus incorrect algorithms

```
BETTERCHANGE( $M, c, d$ )
1   $r \leftarrow M$ 
2  for  $k \leftarrow 1$  to  $d$ 
3     $i_k \leftarrow r/c_k$ 
4     $r \leftarrow r - c_k \cdot i_k$ 
5  return  $(i_1, i_2, \dots, i_d)$ 
```

Correct versus incorrect algorithms

```
BETTERCHANGE( $M, \mathbf{c}, d$ )
1   $r \leftarrow M$ 
2  for  $k \leftarrow 1$  to  $d$ 
3       $i_k \leftarrow r/c_k$ 
4       $r \leftarrow r - c_k \cdot i_k$ 
5  return  $(i_1, i_2, \dots, i_d)$ 
```

- BetterChange is not a correct algorithm. Suppose, $M=40$, $\mathbf{c}=(25, 20, 10, 5, 1)$.
- BetterChange solution: $(1, 0, 1, 1, 0)$
- Optimal solution: $(0, 2, 0, 0, 0)$

Brute force

```
BRUTEFORCECHANGE( $M, \mathbf{c}, d$ )
1   $smallestNumberOfCoins \leftarrow \infty$ 
2  for each  $(i_1, \dots, i_d)$  from  $(0, \dots, 0)$  to  $(M/c_1, \dots, M/c_d)$ 
3       $valueOfCoins \leftarrow \sum_{k=1}^d i_k c_k$ 
4      if  $valueOfCoins = M$ 
5           $numberOfCoins \leftarrow \sum_{k=1}^d i_k$ 
6          if  $numberOfCoins < smallestNumberOfCoins$ 
7               $smallestNumberOfCoins \leftarrow numberOfCoins$ 
8               $bestChange \leftarrow (i_1, i_2, \dots, i_d)$ 
9  return  $(bestChange)$ 
```

BruteForceChance - # iterations

$$\begin{array}{c}
 (0, 0, \dots, 0, 0) \\
 (0, 0, \dots, 0, 1) \\
 (0, 0, \dots, 0, 2) \\
 \vdots \\
 (0, 0, \dots, 0, \frac{M}{c_d}) \\
 (0, 0, \dots, 1, 0) \\
 (0, 0, \dots, 1, 1) \\
 (0, 0, \dots, 1, 2) \\
 \vdots \\
 (0, 0, \dots, 1, \frac{M}{c_d}) \\
 \vdots \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, 0) \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, 1) \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, 2) \\
 \vdots \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, \frac{M}{c_d}) \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}}, 0) \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}}, 1) \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}}, 2) \\
 \vdots \\
 (\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}}, \frac{M}{c_d})
 \end{array}$$

8/29/21

Chapter 2. Algorithms and Complexity

23

23

Fast versus slow algorithms

- Running time as attribute of the computer
 - Supercomputer performs operations $\sim 10^{-9}$ sec
 - # operations an algorithm will perform
 - Running time $\approx$ # operations $\times 10^{-9}$
- Running time as attribute of the algorithm
 - Total number of operations

8/29/21

Chapter 2. Algorithms and Complexity

24

24

Determining number of operations

- USChange will perform 7 operations

USCHANGE(M)

- 1 Give the integer part of $M/25$ quarters to customer.
- 2 Let *remainder* be the remaining amount due the customer.
- 3 Give the integer part of *remainder*/10 dimes to customer.
- 4 Let *remainder* be the remaining amount due the customer.
- 5 Give the integer part of *remainder*/5 nickels to customer.
- 6 Let *remainder* be the remaining amount due the customer.
- 7 Give *remainder* pennies to customer.

- Rather than tediously count every multiplication and addition, we can perform this comparison by gaining a high-level understanding of the growth of each algorithm operation count as the size of the input increases.

BruteForceChange – running time

$(0, 0, \dots, 0, 0)$
 $(0, 0, \dots, 0, 1)$
 $(0, 0, \dots, 0, 2)$
 $\vdots$
 $(0, 0, \dots, 0, \frac{M}{c_d})$
 $(0, 0, \dots, 1, 0)$
 $(0, 0, \dots, 1, 1)$
 $(0, 0, \dots, 1, 2)$
 $\vdots$
 $(0, 0, \dots, 1, \frac{M}{c_d})$
 $\vdots$
 $(\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, 0)$
 $(\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, 1)$
 $(\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, 2)$
 $\vdots$
 $(\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}} - 1, \frac{M}{c_d})$
 $\vdots$
 $(\frac{M}{c_1}, \frac{M}{c_2}, \dots, \frac{M}{c_{d-1}}, \frac{M}{c_d})$

There are roughly

$$\frac{M}{c_1} \cdot \frac{M}{c_2} \cdot \dots \cdot \frac{M}{c_d} = \frac{M^d}{c_1 \cdot c_2 \cdot \dots \cdot c_d}$$

Exponential algorithm

Big O-Notation

- Definition: Let f be a function of n . The term “**asymptotic**” means the study of the function f as n becomes larger and larger without bound.
- Definition: A function $f(x)$ is $O(g(x))$ if there are positive real constants c and x_0 such that $f(x) \leq cg(x)$ for all values $x \geq x_0$.

8/29/21

Chapter 2. Algorithms and Complexity

27

27

Big O-Notation

Function $g(n)$	Growth rate of $f(n)$
$g(n) = 1$	The growth rate is constant and so does not depend on n , the size of the problem.
$g(n) = \log_2 n$	The growth rate is a function of $\log_2 n$. Because a logarithm function grows slowly, the growth rate of the function f is also slow.
$g(n) = n$	The growth rate is linear. The growth rate of f is directly proportional to the size of the problem.
$g(n) = n * \log_2 n$	The growth rate is faster than the linear algorithm.
$g(n) = n^2$	The growth rate of such functions increases rapidly with the size of the problem. The growth rate is quadrupled when the problem size is doubled.
$g(n) = 2^n$	The growth rate is exponential. The growth rate is squared when the problem size is doubled.

8/29/21

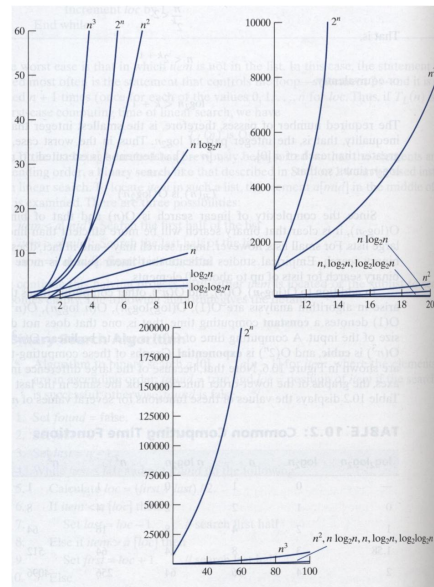
Chapter 2. Algorithms and Complexity

28

28

Functions comparison

- A comparison of common computing times.



8/29/21

Chapter 2. Algorithms and Complexity

29

29

Computing times comparison

Statement	# of Times Executed
$\log_2 \log_2 n$	3 microseconds
$\log_2 n$	8 microseconds
n	.25 milliseconds
$n \log_2 n$	2 milliseconds
n^2	65 milliseconds
n^3	27 seconds
2^n	3.7×10^{61} centuries

Computing times for $n=256$. Suppose each instruction can be executed in one microsecond.

8/29/21

Chapter 2. Algorithms and Complexity

30

30

Iterative versus Recursive Algorithms

Sorting Problem:

Sort a list of integers.

Input: A list of n distinct integers $a = (a_1, a_2, \dots, a_n)$.

Output: Sorted list of integers, that is, a reordering $b = (b_1, b_2, \dots, b_n)$ of integers from a such that $b_1 < b_2 < \dots < b_n$.

SELECTIONSORT(a, n)

```

1  for  $i \leftarrow 1$  to  $n - 1$ 
2       $a_j \leftarrow$  Smallest element among  $a_i, a_{i+1}, \dots, a_n$ .
3      Swap  $a_i$  and  $a_j$ 
4  return  $a$ 
```

(7, 92, 87, 1, 4, 3, 2, 6)

(1, 92, 87, 7, 4, 3, 2, 6)

(1, 2, 87, 7, 4, 3, 92, 6)

(1, 2, 3, 7, 4, 87, 92, 6)

(1, 2, 3, 4, 7, 87, 92, 6)

(1, 2, 3, 4, 6, 87, 92, 7)

(1, 2, 3, 4, 6, 7, 92, 87)

(1, 2, 3, 4, 6, 7, 87, 92)

8/29/21

Chapter 2. Algorithms and Complexity

31

31

Iterative Algorithm for Selection Sort

SELECTIONSORT(a, n)

```

1  for  $i \leftarrow 1$  to  $n - 1$ 
2       $j \leftarrow$  INDEXOFMIN( $a, i, n$ )
3      Swap elements  $a_i$  and  $a_j$ 
4  return  $a$ 
```

INDEXOFMIN($array, first, last$)

```

1   $index \leftarrow first$ 
2  for  $k \leftarrow first + 1$  to  $last$ 
3      if  $array_k < array_{index}$ 
4           $index \leftarrow k$ 
5  return  $index$ 
```

(7, 92, 87, 1, 4, 3, 2, 6)

(1, 92, 87, 7, 4, 3, 2, 6)

(1, 2, 87, 7, 4, 3, 92, 6)

(1, 2, 3, 7, 4, 87, 92, 6)

(1, 2, 3, 4, 7, 87, 92, 6)

(1, 2, 3, 4, 6, 87, 92, 7)

(1, 2, 3, 4, 6, 7, 92, 87)

(1, 2, 3, 4, 6, 7, 87, 92)

8/29/21

Chapter 2. Algorithms and Complexity

32

32

Recursive algorithms

- **Recursive algorithm**
 - Algorithm that finds the solution to a given problem by reducing the problem to smaller versions of itself
 - Has one or more base cases
 - Implemented using recursive functions
- **Recursive function**
 - function that calls itself
- **Base case**
 - Case in recursive definition in which the solution is obtained directly
 - Stops the recursion

8/29/21

Chapter 2. Algorithms and Complexity

33

33

Recursive Algorithm for Selection Sort

```
RECURSIVESELECTIONSORT(a, first, last)
1  if first < last
2    index ← INDEXOFMIN(a, first, last)
3    Swap afirst with aindex
4    a ← RECURSIVESELECTIONSORT(a, first + 1, last)
5  return a
```

(7, 92, 87, 1, 4, 3, 2, 6)
(1, 92, 87, 7, 4, 3, 2, 6)
(1, 2, 87, 7, 4, 3, 92, 6)
(1, 2, 3, 7, 4, 87, 92, 6)
(1, 2, 3, 4, 7, 87, 92, 6)
(1, 2, 3, 4, 6, 87, 92, 7)
(1, 2, 3, 4, 6, 7, 92, 87)
(1, 2, 3, 4, 6, 7, 87, 92)

8/29/21

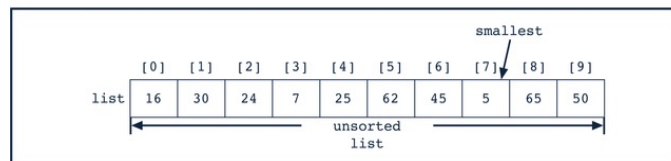
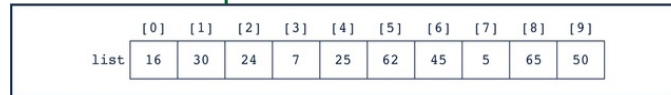
Chapter 2. Algorithms and Complexity

34

34

Running time analysis

Selection Sort Example:

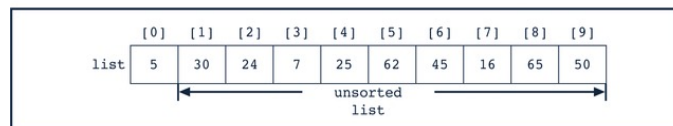
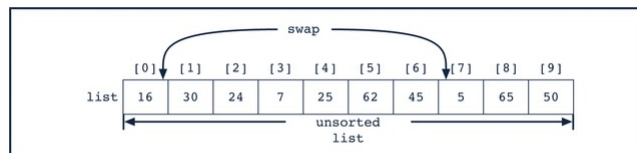


8/29/21

Chapter 2. Algorithms and Complexity

35

35



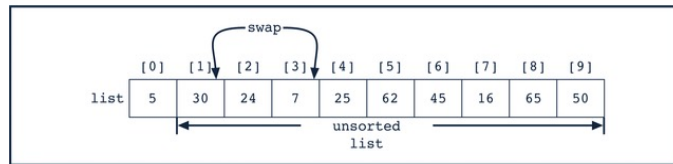
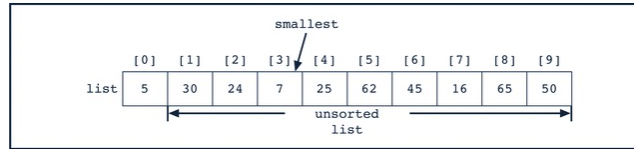
']

8/29/21

Chapter 2. Algorithms and Complexity

36

36



8/29/21

Chapter 2. Algorithms and Complexity

37

37

Analysis: Selection Sort

By analyzing the number of key comparisons, we see that selection sort is an $O(n^2)$ algorithm:

$$\begin{aligned}
 (n-1) + (n-2) + \dots + 2 + 1 &= \frac{n(n-1)}{2} \\
 &= \frac{1}{2}n^2 - \frac{1}{2}n \\
 &= \frac{1}{2}n^2 + O(n) \\
 &= O(n^2).
 \end{aligned}$$

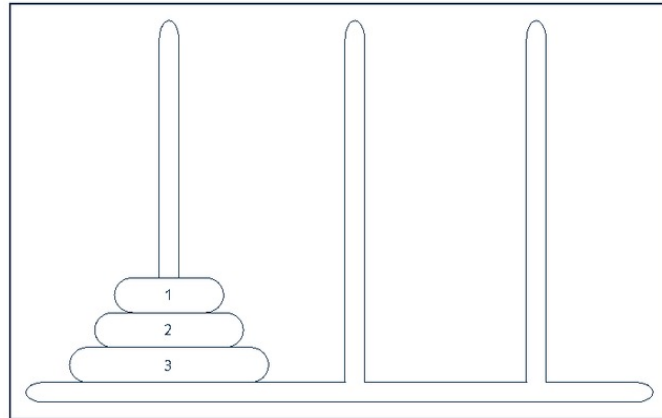
8/29/21

Chapter 2. Algorithms and Complexity

38

38

Towers of Hanoi Problem with 3 Disks



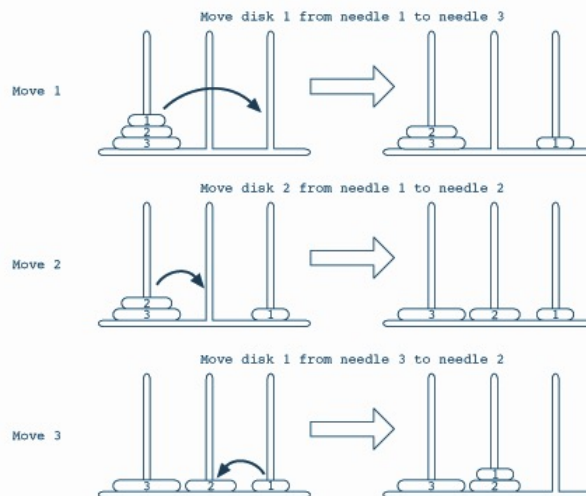
8/29/21

Chapter 2. Algorithms and Complexity

39

39

Towers of Hanoi: 3 Disk Solution



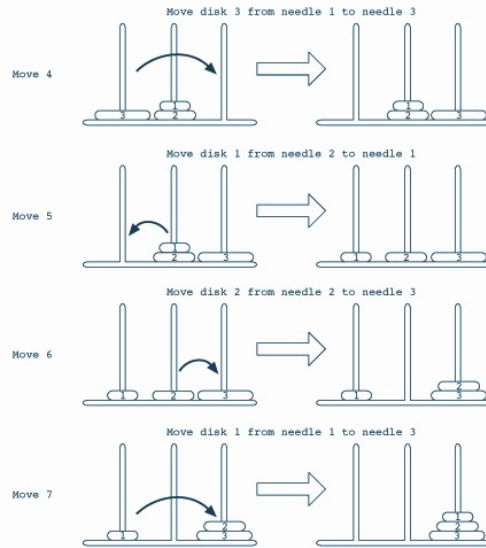
8/29/21

Chapter 2. Algorithms and Complexity

40

40

Towers of Hanoi: 3 Disk Solution

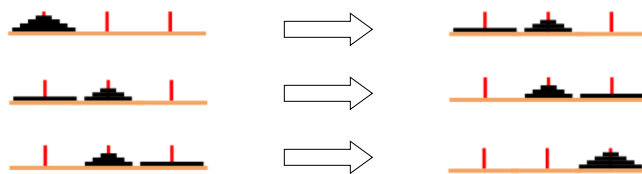


8/29/21

Chapter 2. Algorithms and Complexity

41

Towers of Hanoi: 4 Disk Solution



8/29/21

Chapter 2. Algorithms and Complexity

42

Towers of Hanoi: Recursive Algorithm

```
HanoiTowers(n, needle1, needle3, needle2)
{
    if(count > 0)
    {
        moveDisks(n-1, needle1, needle2, needle3)
        output "Move disk n from needle1 to needle3"
        moveDisks(n-1, needle2, needle3, needle1)
    }
}
```

8/29/21

Chapter 2. Algorithms and Complexity

43

43

Fibonacci Problem

Fibonacci Problem:

Calculate the n th Fibonacci number.

Input: An integer n .

Output: The n th Fibonacci number $F_n = F_{n-1} + F_{n-2}$ (with $F_1 = F_2 = 1$).

RECURSIVEFIBONACCI(n)

```
1 if  $n = 1$  or  $n = 2$ 
2   return 1
3 else
4    $a \leftarrow \text{RECURSIVEFIBONACCI}(n-1)$ 
5    $b \leftarrow \text{RECURSIVEFIBONACCI}(n-2)$ 
6   return  $a + b$ 
```

FIBONACCI(n)

```
1  $F_1 \leftarrow 1$ 
2  $F_2 \leftarrow 1$ 
3 for  $i \leftarrow 3$  to  $n$ 
4    $F_i \leftarrow F_{i-1} + F_{i-2}$ 
5 return  $F_n$ 
```

8/29/21

Chapter 2. Algorithms and Complexity

44

44

Backtracking

- Attempts to find solutions to a problem by constructing partial solutions
- Makes sure that any partial solution does not violate the problem requirements
- Tries to extend partial solution towards completion

8/29/21

Chapter 2. Algorithms and Complexity

45

45

Backtracking (cont.)

- If it is determined that partial solution would not lead to solution
 - partial solution would end in dead end
 - algorithm backs up by removing the most recently added part and then tries other possibilities

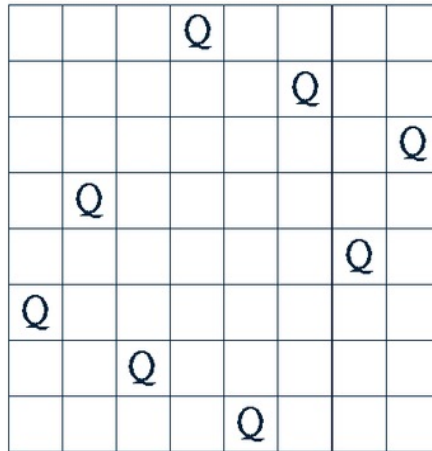
8/29/21

Chapter 2. Algorithms and Complexity

46

46

Solution to 8-Queens Puzzle



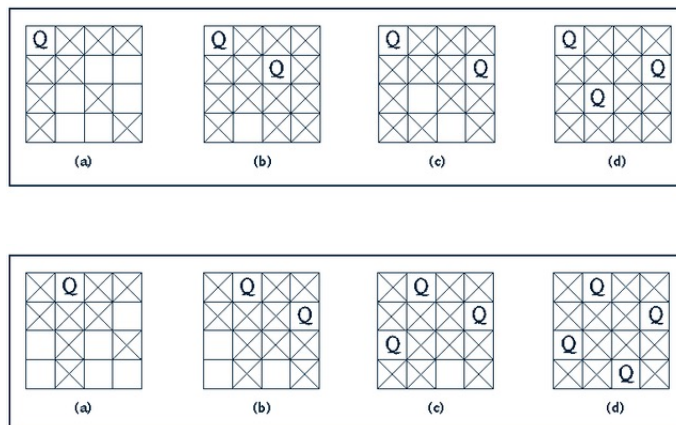
8/29/21

Chapter 2. Algorithms and Complexity

47

47

4-Queens Puzzle



8/29/21

Chapter 2. Algorithms and Complexity

48

48

Algorithm design techniques

The Ringing Telephone problem



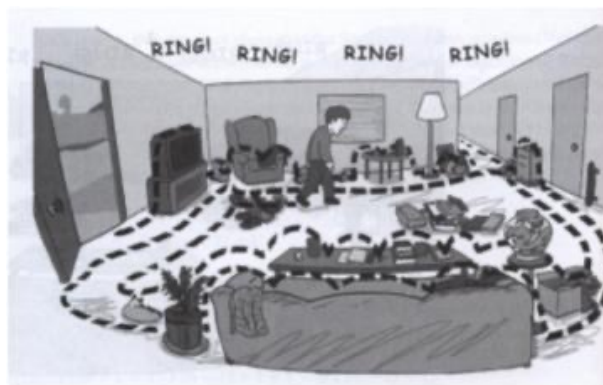
8/29/21

Chapter 2. Algorithms and Complexity

49

49

Exhaustive search (brute force)



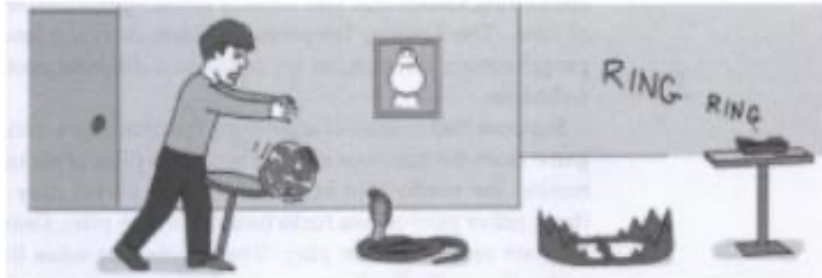
8/29/21

Chapter 2. Algorithms and Complexity

50

50

Greedy search



8/29/21

Chapter 2. Algorithms and Complexity

51

51

Algorithm design techniques

- Branch-and-Bound
- Machine Learning
- Randomized



8/29/21

Chapter 2. Algorithms and Complexity

52

52

Dynamic programming

The 10+10 rocks game

$$R_{i,j} = \begin{cases} W, & \text{if Player 1 can always win the } i+j \text{ rocks game} \\ L, & \text{otherwise} \end{cases}$$

	0	1	2	3	4	5	6	7	8	9	10
0		W									
1	W		W								
2											
3											
4											
5											
6											
7											
8											
9											
10											

8/29/21

Chapter 2. Algorithms and Complexity

53

53

“winning recipe” for the 10+10 game

In the (2,0) case, the only move that Player 1 can make leads to the (1,0) case, which is a winning position for his opponent.

	0	1	2	3	4	5	6	7	8	9	10
0		W	L								
1	W		W								
2	L	W		L							
3											
4											
5											
6											
7											
8											
9											
10											

	0	1	2	3	4	5	6	7	8	9	10
0		W	L								
1	W		W								
2	L	W		L							
3											
4											
5											
6											
7											

In the (2,1) case, Player 1 can make 3 different moves that lead respectively to the games (1,1), (2,0), or (1,0). One of these cases, (2,0), leads to a losing position for his opponent and therefore (2,1) is a winning position.

8/29/21

Chapter 2. Algorithms and Complexity

54

54

Rocks algorithm

```

ROCKS( $n, m$ )
1   $R_{0,0} = L$ 
2  for  $i \leftarrow 1$  to  $n$ 
3    if  $R_{i-1,0} = W$ 
4       $R_{i,0} \leftarrow L$ 
5    else
6       $R_{i,0} \leftarrow W$ 
7    for  $j \leftarrow 1$  to  $m$ 
8      if  $R_{0,j-1} = W$ 
9         $R_{0,j} \leftarrow L$ 
10     else
11        $R_{0,j} \leftarrow W$ 
12  for  $i \leftarrow 1$  to  $n$ 
13    for  $j \leftarrow 1$  to  $m$ 
14      if  $R_{i-1,j-1} = W$  and  $R_{i,j-1} = W$  and  $R_{i-1,j} = W$ 
15         $R_{i,j} \leftarrow L$ 
16      else
17         $R_{i,j} \leftarrow W$ 
18  return  $R_{n,m}$ 

```

	0	1	2	3	4	5	6	7	8	9	10
0		W	L	W	L	W	L	W	L	W	L
1	W	W	W	W	W	W	W	W	W	W	W
2	L	W	L	W	L	W	L	W	L	W	L
3	W	W	W	W	W	W	W	W	W	W	W
4	L	W	L	W	L	W	L	W	L	W	L
5	W	W	W	W	W	W	W	W	W	W	W
6	L	W	L	W	L	W	L	W	L	W	L
7	W	W	W	W	W	W	W	W	W	W	W
8	L	W	L	W	L	W	L	W	L	W	L
9	W	W	W	W	W	W	W	W	W	W	W
10	L	W	L	W	L	W	L	W	L	W	L

8/29/21

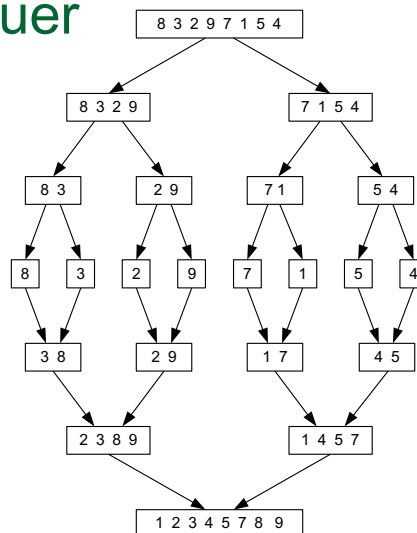
Chapter 2. Algorithms and Complexity

55

55

Divide-and-Conquer

- Merge sort
 - Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
 - Sort arrays B and C recursively
 - Sorted arrays B and C into array A



8/29/21

Chapter 2. Algorithms and Complexity

56

56

Tractable versus intractable problems

- Traveling salesman problem:

Find the shortest path through a set of cities, visiting each city only one time.

- **Input:** A map of cities, roads between the cities, and distances along the roads.
- **Output:** A sequence of roads that will be take a salesman through every city on the map, such that he will visit each city exactly once, and will travel the shortest total distance.

Polynomial ————— NP-Complete ————— Exponential